Graph-Based Product Recommendation Engine

Dijkstra's Algorithm — Full Implementation Plan

AI-Integrated Ecommerce Platform | e22-co2060

Module	Product Recommendation Engine
Algorithm	Dijkstra's Shortest Path (NetworkX)
Tech Stack	Python · FastAPI · Node.js · React · MongoDB
Version	1.0
Status	Ready for Implementation

Table of Contents

1. Executive Summary
2. System Architecture Overview
3. Algorithm Design — Dijkstra's Shortest Path
4. Graph Model Specification
5. Phase 1 — Graph Data Pipeline (AI-ML / Python)
6. Phase 2 — FastAPI Recommendation Service
7. Phase 3 — Express.js Backend Integration
8. Phase 4 — React Frontend Integration
9. Phase 5 — Graph Rebuild Automation
10. Data Flow & Sequence Diagrams
11. File Structure
12. Environment Variables
13. Edge Cases & Fallback Strategy
14. Performance & Scalability
15. Testing Strategy
16. Implementation Checklist

1. Executive Summary

This document provides the complete implementation plan for a Graph-Based Product Recommendation Engine integrated into the AI-Integrated Ecommerce Platform. The engine models the product catalog as a weighted undirected graph and applies Dijkstra's shortest-path algorithm to surface the most closely related products to any item a user is currently viewing.

Unlike traditional collaborative filtering or content-based approaches that operate on flat feature vectors, the graph-based model captures rich, transitive product relationships: products A and C may be strongly related because both are frequently bought with B, even if A and C have never appeared in the same order. Dijkstra traverses these multi-hop connections and returns a ranked list ordered by cumulative dissimilarity distance.

Key Business Outcomes

- ↑ **Average Order Value** — recommend complementary products at checkout.
- ↑ **Discovery Rate** — surface long-tail products hidden from search results.
- ↑ **Session Depth** — keep users navigating via 'Customers Also Bought' chains.
- ↓ **Bounce Rate** — personalised context-aware recommendations reduce exits.
- **Real-Time Graph Traversal** — sub-millisecond in-memory Dijkstra on cached NetworkX graph.

2. System Architecture Overview

The recommendation engine is a fully additive module — it does not modify any existing backend routes, models, or frontend pages. It adds three new layers:

2.1 High-Level Request Flow

```
┌──────────────────────────────────────────────────────────────────────┐
│                    USER VIEWS PRODUCT PAGE                           │
└──────────────────────────────────────────────────────────────────────┘
                            │  GET /api/recommendations/:productId
                            ▼
┌──────────────────────────────────────────────────────────────────────┐
│              Express.js Backend  (Node.js / port 3000)               │
│   recommendationRouter → recommendationController                    │
│   recommendationService  →  axios proxy to FastAPI                   │
└──────────────────────────────────────────────────────────────────────┘
                            │  GET /recommend/:productId?top_n=8
                            ▼
┌──────────────────────────────────────────────────────────────────────┐
│              FastAPI  AI-ML Service  (Python / port 8001)            │
│   api.py  →  graph_cache  →  dijkstra_recommender                   |
│   product_graph (NetworkX)  +  weight_calculator                     │
└──────────────────────────────────────────────────────────────────────┘
                            │  Graph built from MongoDB data
                            ▼
┌──────────────────────────────────────────────────────────────────────┐
│        MongoDB Atlas  (orders + products + productVariants)          │
└──────────────────────────────────────────────────────────────────────┘
                            │  JSON: [{productId, name, score, ...}]
                            ▼
┌──────────────────────────────────────────────────────────────────────┐
│    React Frontend  (Vite / port 5173)                                │
│    useRecommendations hook  →  ProductRecommendations.jsx            │
└──────────────────────────────────────────────────────────────────────┘
```

2.2 Component Responsibilities

Component	Technology	Port	Responsibility
React Frontend	React 19 + Vite + Tailwind CSS	5173	Renders recommendation cards, manages fetch state
Express Backend	Node.js + Express 5	3000	Auth, proxies recommendation requests to FastAPI
FastAPI AI-ML Service	Python + FastAPI + NetworkX	8001	Graph construction, Dijkstra execution, caching
MongoDB Atlas	MongoDB (Mongoose ODM)	Cloud	Source of truth: orders, products, variants

3. Algorithm Design — Dijkstra's Shortest Path

3.1 Why Dijkstra for Recommendations?

Dijkstra's algorithm computes the shortest cumulative weighted path from a source node to all reachable nodes in a non-negative weighted graph. Applied to our product graph:

- The graph is sparse (most products share edges with only a few others), so Dijkstra's O((V + E) log V) heap-based implementation is highly efficient.
- Edge weights represent dissimilarity — lower means more closely related — which maps naturally to Dijkstra's 'minimize cost' objective.
- Multi-hop relationships are captured: a product can be recommended not just because it was bought with the source, but because it is transitively connected through a chain of related products.
- Unlike matrix factorization, no model training is required. The graph is rebuilt from raw order data and product attributes on a schedule.

3.2 Mathematical Definition

Graph G = (V, E, w) where:

```
V  =  { p₁, p₂, ..., pₙ }          (all products in catalog)
E  =  { (pᵢ, pⱼ) | relationship exists between pᵢ and pⱼ }
w  :  E → ℝ⁺                          (dissimilarity weight, lower = more similar)

Dijkstra(G, source=p_current):
  dist[v] = min cumulative weight from p_current to v
  result  = sorted(dist.items(), key=lambda x: x[1])[:top_n]
```

3.3 Edge Weight Formula

Two signals contribute to each edge weight:

Signal 1 — Co-Purchase Weight (α = 0.6)

```
co_purchase_weight(A, B)  =  α / (co_purchase_count(A, B) + 1)

co_purchase_count(A, B)  =  number of orders containing BOTH product A and
product B

Example:  A and B bought together 10 times  →  0.6 / 11  =  0.0545
          A and B bought together  1 time   →  0.6 /  2  =  0.300
          A and B never co-purchased        →  edge from attribute signal only
```

Signal 2 — Attribute Similarity Weight (β = 0.4)

```
features(P)  =  { category } ∪ { brand } ∪ set(specs.keys())

jaccard_similarity(A, B)  =  |features(A) ∩ features(B)| / |features(A) ∪
features(B)|

attribute_weight(A, B)    =  β × (1 - jaccard_similarity(A, B))

Example:  A = {electronics, samsung, RAM, storage}
          B = {electronics, samsung, RAM, color}
```

```
            intersection = {electronics, samsung, RAM}  →  |∩| = 3
            union        = {electronics, samsung, RAM, storage, color}  →  |∪| = 5
            jaccard_sim  = 3/5 = 0.60   →   weight = 0.4 × (1-0.60) = 0.16
```

Combined Edge Weight

```
  w(A, B)  =  co_purchase_weight(A, B)  +  attribute_weight(A, B)

  Special cases:
    - No co-purchase data:  α = 0.0,  β = 1.0  (attribute only)
    - New product (no orders & sparse attributes):  connect to category cluster
centroid
```

4. Graph Model Specification

4.1 Node Schema

```
Node ID  :  productId  (MongoDB ObjectId string)
Node metadata stored as NetworkX node attributes:
  {
    'productName'  : str,
    'category'     : str,
    'brand'        : str,
    'image'        : str,
    'minPrice'     : float,
    'specs'        : dict,
    'soldCount'    : int,
  }
```

4.2 Edge Schema

```
Edge (productA_id, productB_id)  — undirected
Edge attributes:
  {
    'weight'            : float,   # combined dissimilarity score
    'co_purchase_count' : int,     # raw frequency from orders
    'attribute_sim'     : float,   # jaccard similarity
    'relationship_type' : str,     # 'co_purchase' | 'attribute' | 'both'
  }
```

4.3 Graph Properties

Property	Value	Reason
Type	Undirected	A→B and B→A relationships are symmetric
Weights	Positive $\mathbb{R}^+$	Required by Dijkstra (no negative weights)
Self-loops	None	A product cannot be related to itself
Persistence	.gpickle file	Avoids rebuilding on every API request
In-memory	Singleton	Loaded once at FastAPI startup
Rebuild	Every 24h	Captures new orders and product additions

5. Phase 1 — Graph Data Pipeline (AI-ML / Python)

All files live under AI-ML/recommendation/

5.1 graph_builder.py — MongoDB Data Extractor

Connects to MongoDB Atlas and extracts two raw datasets needed to construct the graph.

- **Products Query:** Load all products and their metadata (category, brand, specs).
- **Orders Query:** Load all delivered/confirmed orders and flatten their items arrays to extract product pairs.
- **Pair Extraction:** For each order containing N items, generate C(N,2) product pairs → increment co-purchase counter.

```
  # Pseudo-code
  co_purchase = defaultdict(int)
  for order in orders_collection.find({'status': {'$in':
['confirmed','delivered']}}):
      product_ids = [str(item['productId']) for item in order['items']]
      for i in range(len(product_ids)):
          for j in range(i+1, len(product_ids)):
              key = tuple(sorted([product_ids[i], product_ids[j]]))
              co_purchase[key] += 1

  # Returns: co_purchase dict, products list
```

5.2 weight_calculator.py — Dissimilarity Scoring

Computes the final edge weight for every product pair.

```
  ALPHA = 0.6   # co-purchase signal weight
  BETA  = 0.4   # attribute signal weight

  def co_purchase_weight(count, alpha=ALPHA):
      return alpha / (count + 1)

  def attribute_weight(prod_a, prod_b, beta=BETA):
      feats_a = build_feature_set(prod_a)
      feats_b = build_feature_set(prod_b)
      if not feats_a and not feats_b:
          return beta
      intersection = len(feats_a & feats_b)
      union        = len(feats_a | feats_b)
      jaccard_sim  = intersection / union if union > 0 else 0
      return beta * (1 - jaccard_sim)

  def combined_weight(count, prod_a, prod_b):
      cw = co_purchase_weight(count) if count > 0 else 0
      aw = attribute_weight(prod_a, prod_b)
      return cw + aw
```

5.3 product_graph.py — NetworkX Graph Manager

Builds the undirected weighted graph in memory and persists it to disk as product_graph.gpickle for fast reloading.

```python
import networkx as nx
import pickle

def build_graph(products, co_purchase) -> nx.Graph:
    G = nx.Graph()
    # Add nodes
    for p in products:
        G.add_node(str(p['_id']),
            productName=p['productName'],
            category=p['category'],
            brand=p.get('brand',''),
            image=p.get('image',''),
            minPrice=p.get('minPrice', 0.0),
            specs=p.get('specs', {}),
        )
    # Add co-purchase edges
    for (pid_a, pid_b), count in co_purchase.items():
        w = combined_weight(count, product_map[pid_a], product_map[pid_b])
        G.add_edge(pid_a, pid_b, weight=w,
            co_purchase_count=count, relationship_type='co_purchase')
    # Add attribute-only edges for same-category products (no co-purchase)
    add_attribute_edges(G, products, product_map)
    return G

def save_graph(G, path='product_graph.gpickle'):
    with open(path, 'wb') as f: pickle.dump(G, f)

def load_graph(path='product_graph.gpickle') -> nx.Graph:
    with open(path, 'rb') as f: return pickle.load(f)
```

5.4 dijkstra_recommender.py — Core Recommendation Engine

```python
import networkx as nx

def get_recommendations(G: nx.Graph, product_id: str,
                        top_n: int = 8, cutoff: float = 5.0):
    if product_id not in G:
        return fallback_by_category(G, product_id, top_n)

    # Dijkstra: returns {node: distance} for all reachable nodes
    distances = nx.single_source_dijkstra_path_length(
        G, source=product_id, cutoff=cutoff
    )

    # Sort by distance ascending, exclude the source product itself
    ranked = sorted(
        [(node, dist) for node, dist in distances.items() if node != product_id],
        key=lambda x: x[1]
    )[:top_n]

    # Enrich with product metadata from node attributes
    results = []
    for node_id, dist in ranked:
        attrs = G.nodes[node_id]
        edge_data = G.edges.get((product_id, node_id), {})
        results.append({
```

```
            'productId'         : node_id,
            'productName'       : attrs.get('productName', ''),
            'image'             : attrs.get('image', ''),
            'category'          : attrs.get('category', ''),
            'minPrice'          : attrs.get('minPrice', 0.0),
            'dissimilarityScore': round(dist, 4),
            'relationshipType'  : edge_data.get('relationship_type',
'similar_to'),
        })
    return results

def fallback_by_category(G, product_id, top_n):
    # Returns nodes with same category if source not in graph
    ...
```

5.5 graph_cache.py — Singleton Cache & Rebuild Scheduler

Maintains a single in-memory graph instance. Automatically rebuilds the graph if the persisted file is older than GRAPH_REBUILD_INTERVAL_HOURS (default 24h).

```
_graph_instance = None
_last_built     = None

def get_graph() -> nx.Graph:
    global _graph_instance, _last_built
    if _graph_instance is None or _is_stale():
        _graph_instance = _rebuild()   # rebuild in background thread
        _last_built = datetime.utcnow()
    return _graph_instance

def _is_stale() -> bool:
    if _last_built is None: return True
    delta = datetime.utcnow() - _last_built
    return delta.total_seconds() > REBUILD_INTERVAL_SECONDS

def _rebuild() -> nx.Graph:
    products, co_purchase = graph_builder.extract_data()
    G = product_graph.build_graph(products, co_purchase)
    product_graph.save_graph(G)
    return G
```

6. Phase 2 — FastAPI Recommendation Service

6.1 API Endpoints

Method	Endpoint	Auth	Description
GET	/recommend/{product_id}	Public	Dijkstra top-N recommendations for product
GET	/recommend/{product_id}?top_n=N	Public	Override default top_n (default=8, max=20)
GET	/recommend/{product_id}/path/{target_id}	Public	Shortest path between two products with waypoints
POST	/recommend/graph/rebuild	Admin	Force full graph rebuild from MongoDB
GET	/recommend/graph/stats	Public	Node count, edge count, last rebuild timestamp

6.2 Response Schema (Pydantic)

```
class RecommendedProduct(BaseModel):
    productId          : str
    productName        : str
    image              : str
    category           : str
    minPrice           : float
    dissimilarityScore : float   # Dijkstra cumulative distance (lower = more
similar)
    relationshipType   : str     # 'frequently_bought_together' | 'similar_to' |
'both'

class RecommendationResponse(BaseModel):
    sourceProductId   : str
    recommendations   : List[RecommendedProduct]
    totalFound        : int
    graphStats        : dict     # { nodeCount, edgeCount, lastBuilt }
    processingTimeMs  : float

class PathResponse(BaseModel):
    sourceProductId   : str
    targetProductId   : str
    path              : List[str]       # ordered list of product IDs
    totalDistance     : float
    hops              : int
```

6.3 Startup Event

```
# In AI-ML/app.py
@app.on_event('startup')
async def startup_event():
    # Load or build graph on service start
    graph_cache.get_graph()
    logger.info('Product recommendation graph loaded.')

# Register recommendation router
app.include_router(recommendation_router, prefix='/recommend')
```

7. Phase 3 — Express.js Backend Integration

7.1 recommendationService.js

```
  // Backend/services/recommendationService.js
  const axios = require('axios');
  const RECO_URL = process.env.RECOMMENDATION_SERVICE_URL;  //
http://localhost:8001

  async function getRecommendations(productId, topN = 8) {
    try {
      const { data } = await axios.get(
        `${RECO_URL}/recommend/${productId}`,
        { params: { top_n: topN }, timeout: 5000 }
      );
      return data;
    } catch (err) {
      console.error('[RecommendationService] FastAPI unavailable:', err.message);
      return { recommendations: [], totalFound: 0 };  // graceful degradation
    }
  }

  async function getProductPath(productId, targetId) {
    const { data } = await axios.get(
      `${RECO_URL}/recommend/${productId}/path/${targetId}`
    );
    return data;
  }

  module.exports = { getRecommendations, getProductPath };
```

7.2 recommendationController.js

```
  // Backend/controllers/recommendationController.js
  const { getRecommendations, getProductPath } =
require('../services/recommendationService');

  exports.getRecommendations = async (req, res) => {
    const { productId } = req.params;
    const topN = parseInt(req.query.top_n) || 8;
    const data = await getRecommendations(productId, topN);
    res.json(data);
  };

  exports.getProductPath = async (req, res) => {
    const { productId, targetId } = req.params;
    const data = await getProductPath(productId, targetId);
    res.json(data);
  };

  exports.rebuildGraph = async (req, res) => {
    // Admin only — verified by JWT middleware
    await axios.post(`${RECO_URL}/recommend/graph/rebuild`);
    res.json({ message: 'Graph rebuild triggered successfully.' });
  };
```

7.3 recommendationRouter.js

```
// Backend/routers/recommendationRouter.js
const router = require('express').Router();
const ctrl   = require('../controllers/recommendationController');
const { verifyAdminToken } = require('../middleware/authMiddleware');

router.get('/:productId',                    ctrl.getRecommendations);
router.get('/:productId/path/:targetId',     ctrl.getProductPath);
router.post('/rebuild', verifyAdminToken,    ctrl.rebuildGraph);

module.exports = router;

// Register in app.js:
// app.use('/api/recommendations', recommendationRouter);
```

8. Phase 4 — React Frontend Integration

8.1 useRecommendations.js — Custom Hook

```
// Frontend/src/hooks/useRecommendations.js
import { useState, useEffect } from 'react';
import { fetchRecommendations } from '../services/recommendationService';

export function useRecommendations(productId, topN = 8) {
  const [recommendations, setRecommendations] = useState([]);
  const [loading, setLoading]   = useState(false);
  const [error,   setError]     = useState(null);

  useEffect(() => {
    if (!productId) return;
    // Check session cache first
    const cached = sessionStorage.getItem(`reco_${productId}`);
    if (cached) { setRecommendations(JSON.parse(cached)); return; }

    setLoading(true);
    fetchRecommendations(productId, topN)
      .then(data => {
        setRecommendations(data.recommendations || []);
        sessionStorage.setItem(`reco_${productId}`,
JSON.stringify(data.recommendations));
      })
      .catch(err => setError(err.message))
      .finally(() => setLoading(false));
  }, [productId, topN]);

  return { recommendations, loading, error };
}
```

8.2 ProductRecommendations.jsx — UI Component

Key features of the component:

- Horizontal scrollable card row with smooth CSS scroll-snap
- Each card: product image, name, category badge, min price, 'Add to Cart' CTA
- Relationship badge — 'Bought Together' (green) or 'Similar' (blue) on each card
- Loading state: Tailwind animate-pulse skeleton cards (8 placeholders)
- Empty state: 'No recommendations yet' with a subtle illustration
- Score tooltip: hover shows raw dissimilarity score for developer mode

```
// Frontend/src/components/ProductRecommendations.jsx
import { useRecommendations } from '../hooks/useRecommendations';

export default function ProductRecommendations({ productId }) {
  const { recommendations, loading } = useRecommendations(productId);

  if (loading) return <SkeletonRow />;
  if (!recommendations.length) return null;

  return (
```

```
    <section className='mt-10'>
      <h2 className='text-xl font-bold text-gray-800 mb-4'>
        Customers Also Bought
      </h2>
      <div className='flex gap-4 overflow-x-auto pb-2 scroll-smooth snap-x'>
        {recommendations.map(p => (
          <ProductCard key={p.productId} product={p} />
        ))}
      </div>
    </section>
  );
}
```

8.3 Integration Point — Product Detail Page

```
// In the existing product detail page component (e.g., ProductDetail.jsx)
import ProductRecommendations from '../components/ProductRecommendations';

// Add below the main product info section:
<ProductRecommendations productId={product._id} />
```

9. Phase 5 — Graph Rebuild Automation

9.1 Scheduled Rebuild (Two Options)

Option A — node-cron in Express Backend (Recommended)

```
// Backend/jobs/graphRebuildJob.js
const cron = require('node-cron');   // already in package.json
const axios = require('axios');

// Run every day at 02:00 AM
cron.schedule('0 2 * * *', async () => {
  try {
    await
axios.post(`${process.env.RECOMMENDATION_SERVICE_URL}/recommend/graph/rebuild`);
    console.log('[Cron] Product recommendation graph rebuilt at', new Date());
  } catch (err) {
    console.error('[Cron] Graph rebuild failed:', err.message);
  }
});
```

Option B — APScheduler inside FastAPI

```
# In AI-ML/recommendation/api.py
from apscheduler.schedulers.background import BackgroundScheduler

scheduler = BackgroundScheduler()
scheduler.add_job(graph_cache.force_rebuild, 'cron', hour=2, minute=0)
scheduler.start()
```

9.2 Admin Dashboard Rebuild Button

Add a button in the admin panel that calls POST /api/recommendations/rebuild with the admin JWT token. Show a loading spinner and toast on success/failure.

10. Data Flow & Sequence Diagram

10.1 Graph Build Sequence

```
Scheduler / Admin
     │
     │  POST /recommend/graph/rebuild
     ▼
FastAPI api.py
     │  calls graph_cache.force_rebuild()
     ▼
graph_builder.py
     │  1. query MongoDB: products collection  → products list
     │  2. query MongoDB: orders collection    → flatten items → co_purchase dict
     ▼
weight_calculator.py
     │  for each product pair: compute w = α/(count+1) + β×(1-jaccard)
     ▼
product_graph.py
     │  G = nx.Graph();  add_nodes; add_edges with weights
     │  save_graph(G)  →  product_graph.gpickle
     ▼
graph_cache._graph_instance = G
     │
     ▼
200 OK  { message: 'Graph rebuilt', nodes: N, edges: E }
```

10.2 Recommendation Request Sequence

```
Browser (React)
     │  useRecommendations(productId) hook fires on page load
     │  GET /api/recommendations/:productId
     ▼
Express recommendationController.getRecommendations()
     │  calls recommendationService.getRecommendations(productId, topN)
     │  GET ${RECOMMENDATION_SERVICE_URL}/recommend/:productId?top_n=8
     ▼
FastAPI /recommend/{product_id}
     │  G = graph_cache.get_graph()           ← singleton, no rebuild
     │  distances = nx.single_source_dijkstra_path_length(G, product_id)
     │  ranked = sort(distances)[:8]
     │  enrich each result with node attributes
     ▼
JSON Response: { sourceProductId, recommendations: [...], graphStats }
     ▼
Express forwards response to browser
     ▼
React renders ProductRecommendations.jsx cards
```

11. Complete File Structure

Files to CREATE (all new — nothing existing is modified):

```
AI-ML/
└── recommendation/
    ├── __init__.py
    ├── api.py                    ← FastAPI router (endpoints)
    ├── graph_builder.py          ← MongoDB extraction (orders + products)
    ├── weight_calculator.py      ← Co-purchase & attribute dissimilarity
    ├── product_graph.py          ← NetworkX graph build + persist
    ├── dijkstra_recommender.py   ← Core Dijkstra recommendation engine
    └── graph_cache.py            ← Singleton cache + auto-rebuild

Backend/
├── controllers/
│   └── recommendationController.js
├── routers/
│   └── recommendationRouter.js
├── services/
│   └── recommendationService.js
└── jobs/
    └── graphRebuildJob.js

Frontend/src/
├── services/
│   └── recommendationService.js
├── hooks/
│   └── useRecommendations.js
└── components/
    ├── ProductRecommendations.jsx
    └── RecommendationPath.jsx     ← (optional visual path explorer)
```

Files to MODIFY (minimal changes only):

```
Backend/app.js (or server.js)
  + app.use('/api/recommendations', recommendationRouter);
  + require('./jobs/graphRebuildJob');

AI-ML/app.py
  + app.include_router(recommendation_api.router, prefix='/recommend')
  + @app.on_event('startup') → graph_cache.get_graph()

Backend/.env
  + RECOMMENDATION_SERVICE_URL=http://localhost:8001

AI-ML/.env
  + GRAPH_REBUILD_INTERVAL_HOURS=24
  + RECOMMENDATION_TOP_N=8
  + CO_PURCHASE_WEIGHT_ALPHA=0.6
  + ATTRIBUTE_WEIGHT_BETA=0.4
```

12. Environment Variables

Variable	Location	Default	Description
RECOMMENDATION_SERVICE_URL	Backend .env	http://localhost:8001	Base URL of FastAPI recommendation service
GRAPH_REBUILD_INTERVAL_HOURS	AI-ML .env	24	Hours between automatic graph rebuilds
RECOMMENDATION_TOP_N	AI-ML .env	8	Default number of recommendations returned
CO_PURCHASE_WEIGHT_ALPHA	AI-ML .env	0.6	Weight for co-purchase signal in edge formula
ATTRIBUTE_WEIGHT_BETA	AI-ML .env	0.4	Weight for attribute similarity signal
GRAPH_PICKLE_PATH	AI-ML .env	./product_graph.gpickle	File path for persisted graph
MONGO_URI	AI-ML .env	(existing)	MongoDB Atlas connection string (already set)

13. Edge Cases & Fallback Strategy

Scenario	Detection	Fallback Behaviour
New product — no orders	product_id not in any order.items	Use attribute similarity only (β=1.0, α=0.0). Connect to same-category products.
Product not in graph	product_id not in G.nodes	Return top products from same category sorted by howManyProductsSold.
Isolated node — no edges	G.degree(product_id) == 0	Expand Dijkstra cutoff from 5.0 → 10.0, or return most popular products.
Graph empty / not built	G is None or G.number_of_nodes() == 0	Return documents from existing TopProduct MongoDB collection.
FastAPI service offline	axios timeout / connection refused	Express returns { recommendations: [] }. Frontend shows nothing (no crash).
Graph stale (>24h)	_is_stale() returns True	Background thread rebuilds graph; current stale graph still serves requests.
Disconnected graph components	distances dict is small (<8 nodes)	Combine results from multiple Dijkstra calls across components.

14. Performance & Scalability

14.1 Algorithm Complexity

Operation	Complexity	Notes
Dijkstra (heap-based)	O((V + E) log V)	NetworkX uses heapq; efficient for sparse graphs
Graph build from MongoDB	O(N orders × avg items²)	Run offline/scheduled, not on-request
Jaccard feature distance	O(|features|)	Tiny — max ~20 features per product
Graph pickle load	O(V + E)	One-time at FastAPI startup
In-memory Dijkstra call	< 5 ms	For catalog of 10,000 products, ~200K edges

14.2 Scaling Considerations

- **Up to 10K products** — single NetworkX in-memory graph, no issues.
- **10K–100K products** — partition graph by category; run Dijkstra within-partition first.
- **100K+ products** — migrate to a dedicated graph database (Neo4j / Amazon Neptune).
- **High traffic** — Redis cache recommendation results per productId with 1-hour TTL.
- **Multiple sellers** — graph edges already use productId (not variantId), so seller changes don't require rebuild.

15. Testing Strategy

15.1 Unit Tests (Python / pytest)

Test File	Tests
test_weight_calculator.py	co_purchase_weight(), attribute_weight(), combined_weight() with known inputs
test_product_graph.py	build_graph() produces correct node/edge count; edge weights are positive
test_dijkstra.py	get_recommendations() returns top-N sorted by distance; handles missing node gracefully
test_graph_cache.py	get_graph() returns same instance on repeat calls; _is_stale() triggers rebuild

15.2 Integration Tests

Test	Tool	Expected
GET /recommend/{id} returns 200	pytest + httpx	{ recommendations: [...], totalFound: N }
GET /recommend/{id} with unknown id returns fallback	pytest	Same category products returned
POST /graph/rebuild triggers rebuild	pytest	Graph file timestamp updated
Express /api/recommendations/:id proxies correctly	Jest + supertest	Same data as FastAPI direct call

15.3 Frontend Tests

- useRecommendations hook — mock axios, verify state transitions: loading → recommendations populated.
- ProductRecommendations.jsx — React Testing Library: renders skeleton on loading, renders N cards on data, renders nothing on empty array.

16. Implementation Checklist

Use this checklist to track progress during development.

Phase 1 — Graph Data Pipeline

- ☐ Create AI-ML/recommendation/ module folder with __init__.py
- ☐ Install dependencies: networkx, pymongo, python-dotenv (pip install)
- ☐ Implement graph_builder.py — MongoDB data extraction
- ☐ Implement weight_calculator.py — co-purchase + attribute weights
- ☐ Implement product_graph.py — build, save, load graph
- ☐ Implement dijkstra_recommender.py — Dijkstra + fallback
- ☐ Implement graph_cache.py — singleton + staleness check
- ☐ Write unit tests for all 5 Python modules

Phase 2 — FastAPI Endpoints

- ☐ Create AI-ML/recommendation/api.py with 4 endpoints
- ☐ Define Pydantic response schemas
- ☐ Register router in AI-ML/app.py with /recommend prefix
- ☐ Add startup event to load/build graph
- ☐ Test all endpoints with curl / Postman

Phase 3 — Express.js Backend

- ☐ Create Backend/services/recommendationService.js
- ☐ Create Backend/controllers/recommendationController.js
- ☐ Create Backend/routers/recommendationRouter.js
- ☐ Register router in Backend/app.js
- ☐ Add RECOMMENDATION_SERVICE_URL to Backend/.env
- ☐ Test proxy with supertest

Phase 4 — React Frontend

- ☐ Create Frontend/src/services/recommendationService.js
- ☐ Create Frontend/src/hooks/useRecommendations.js
- ☐ Create Frontend/src/components/ProductRecommendations.jsx
- ☐ Integrate <ProductRecommendations> into product detail page
- ☐ Test loading, data, and empty states
- ☐ (Optional) Create RecommendationPath.jsx visual component

Phase 5 — Automation

- ☐ Create Backend/jobs/graphRebuildJob.js with node-cron
- ☐ Register cron job in Backend/app.js
- ☐ Add admin dashboard 'Rebuild Graph' button + toast feedback
- ☐ Verify graph rebuilds on schedule

Final Validation

- ☐ End-to-end test: view product → recommendations appear on page
- ☐ Verify fallback when FastAPI is offline
- ☐ Performance test: Dijkstra response < 100ms on full catalog
- ☐ Add product_graph.gpickle to .gitignore

Appendix A — New Dependencies to Install

Python (AI-ML)

```
# Add to AI-ML/requirements.txt
networkx>=3.2
pymongo>=4.6
python-dotenv>=1.0   # likely already present
apscheduler>=3.10    # optional: for Option B scheduler
```

Node.js (Backend)

```
# node-cron is already in package.json
# axios is likely already present
# No new npm packages required
```

React (Frontend)

```
# No new npm packages required
# Optional: npm install react-flow-renderer  (for visual path component only)
```